VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

REPORT : LAB

Internet of Thing Application Development

Class: CO3038 - CC01

Lecturer: Phan Văn Sỹ

No.	Full Name	ID Student
1.	**Lê Mạnh Quân**	**2352993**

Ho Chi Minh City, November 2025

Contents

1. Lab 1: Tài khoản và CoreIoT Dashboard

1.1. Tạo tài khoản CoreIoT

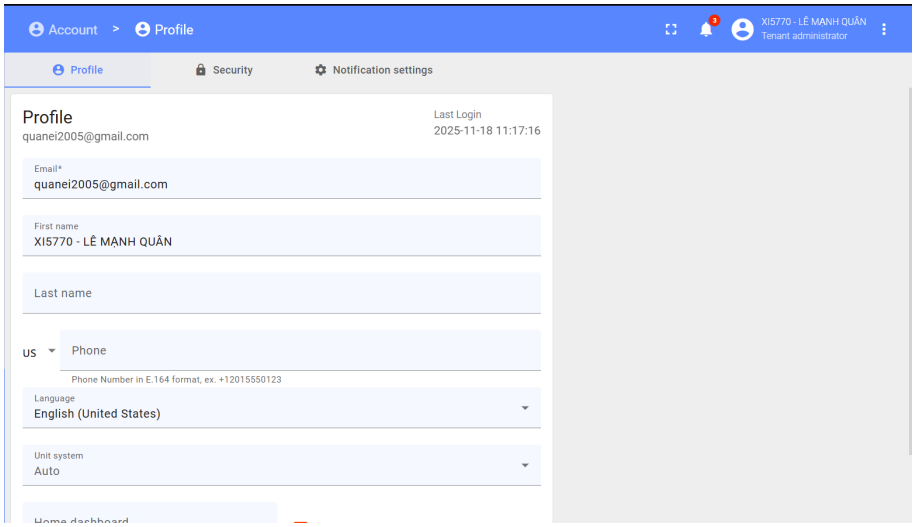

Figure 1: Ảnh minh họa quá trình tạo tài khoản CoreIoT

1.2. Giao diện Dashboard CoreIoT

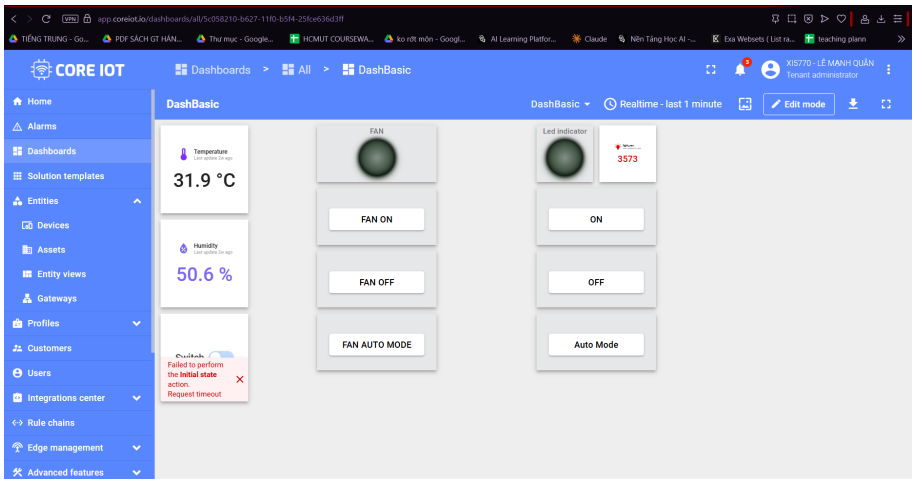

Figure 2: Giao diện Dashboard CoreIoT

1.3. Public dữ liệu device-CoreIoT

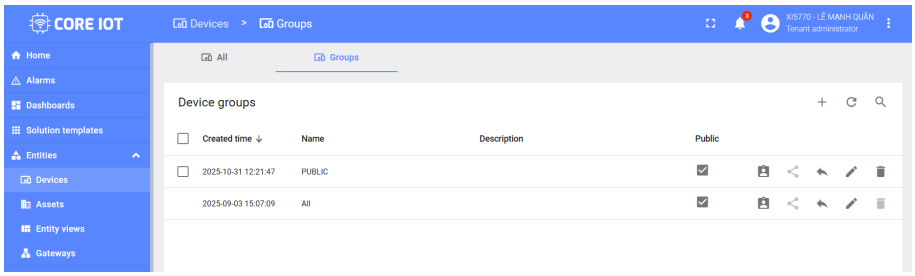

Figure 3: Quá trình publish dữ liệu lên CoreIoT

2. Lab 2: LED Blinky và DHT20 với CoreIoT

2.1. Code điều khiển LED Blinky

```
#include "led_blinky.h"

void led_blinky(void *pvParameters){
  pinMode(LED_GPIO, OUTPUT);

  while(1) {
    digitalWrite(LED_GPIO, HIGH);  // turn the LED ON
    vTaskDelay(1000);
    digitalWrite(LED_GPIO, LOW);  // turn the LED OFF
    vTaskDelay(1000);
  }
}
```

Listing 1: Chương trình LED Blinky

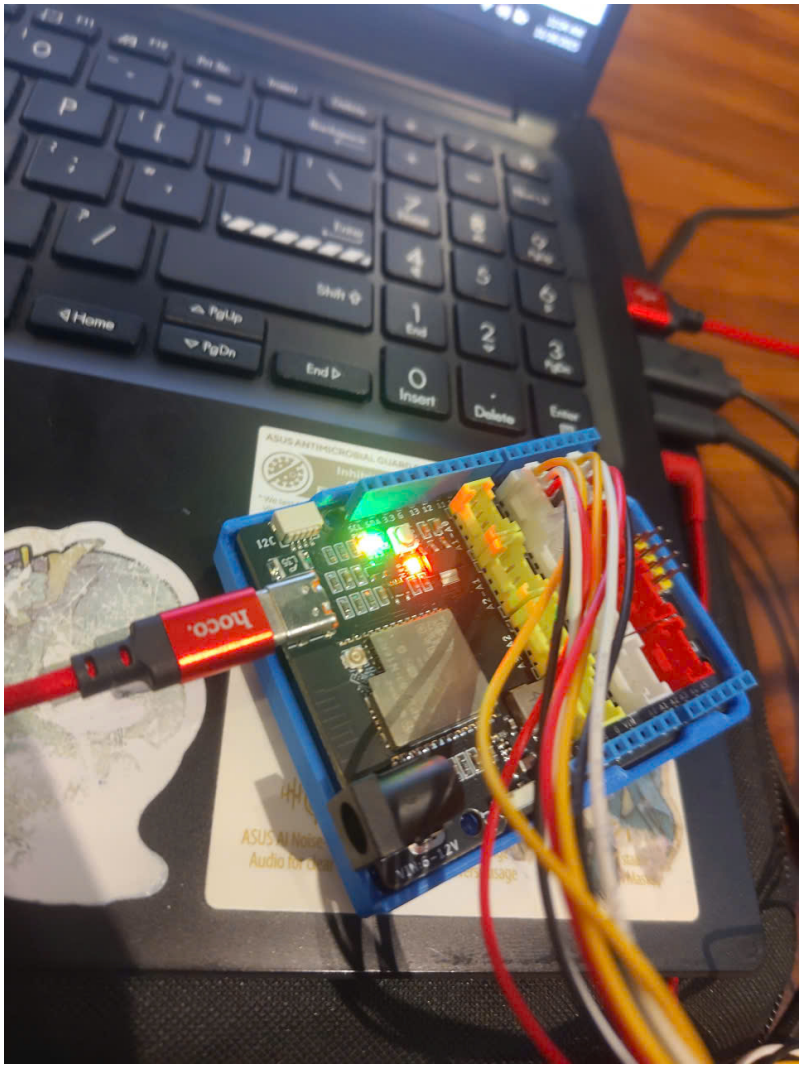

Figure 4: Kết quả LED Blinky trên kit

2.2. Code đọc DHT20 và gửi dữ liệu lên CoreIoT

```
1
2  DHT20 dht20;
3  LiquidCrystal_I2C lcd(0x21, 16, 2);
4
5  void temp_humi_monitor(void *pvParameters) {
6    Wire.begin(SDA_PIN, SCL_PIN);
7    dht20.begin();
8    lcd.init();
9    lcd.backlight();
10
11   char lineBuf[32];
12
13   for (;;) {
```

```
    dht20.read();
    float temperature = dht20.getTemperature();
    float humidity    = dht20.getHumidity();

    if (isnan(temperature) || isnan(humidity)) {
      temperature = humidity = -1;
    }

    glob_temperature = temperature;
    glob_humidity    = humidity;

    Serial.printf("Humidity: %.1f%%  Temp: %.1f C\n", humidity,
        temperature);

    lcd.clear();
    lcd.setCursor(0, 0);
    snprintf(lineBuf, sizeof(lineBuf), "Temp: %4.1f C", temperature
        );
    lcd.print(lineBuf);
    lcd.setCursor(0, 1);
    snprintf(lineBuf, sizeof(lineBuf), "Humid: %4.1f %%", humidity)
        ;
    lcd.print(lineBuf);

    vTaskDelay(pdMS_TO_TICKS(5000));
  }
}
```

Listing 2: Đọc dữ liệu từ DHT20 và gửi lên CoreIoT

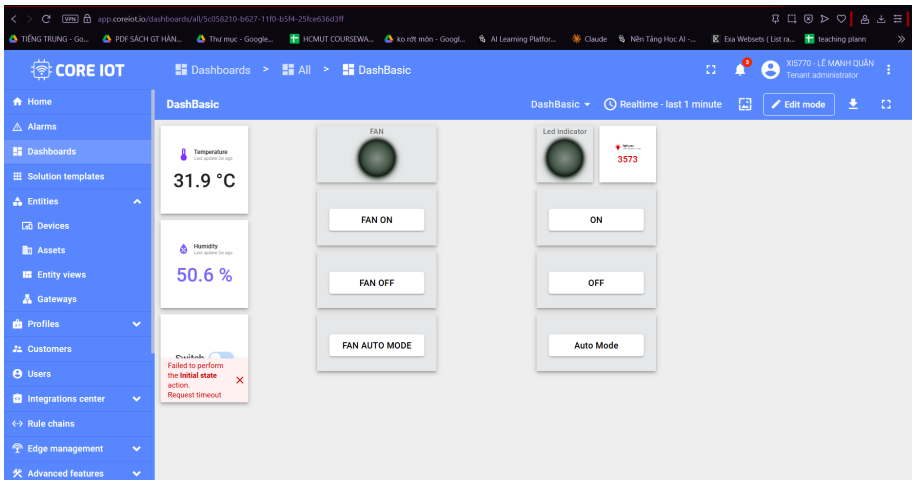

Figure 5: Dữ liệu cảm biến DHT20 hiển thị trên CoreIoT

3. Lab 3: Webserver và TinyML

3.1. ESP32 Connect

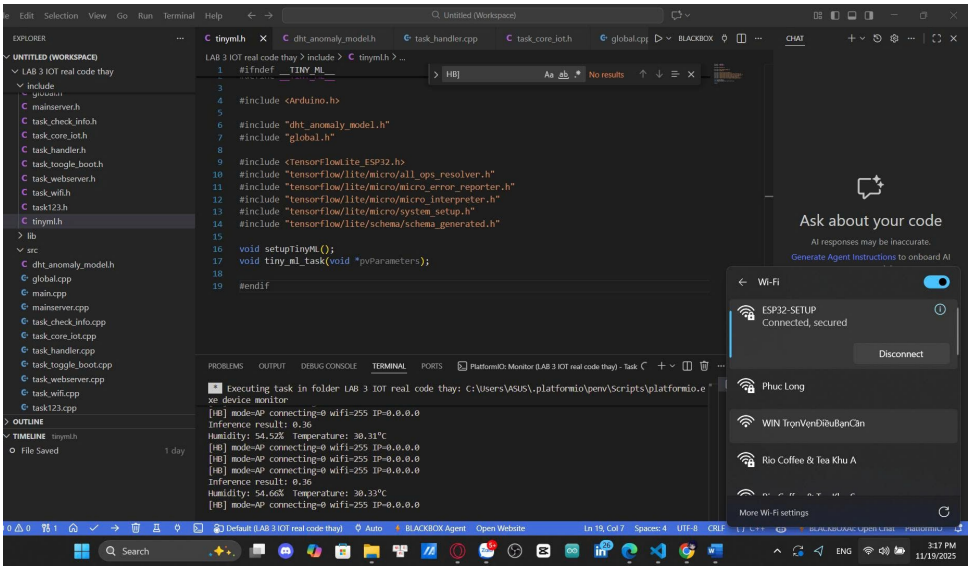

Figure 6: Kết nối với esp32

3.2. Webserver

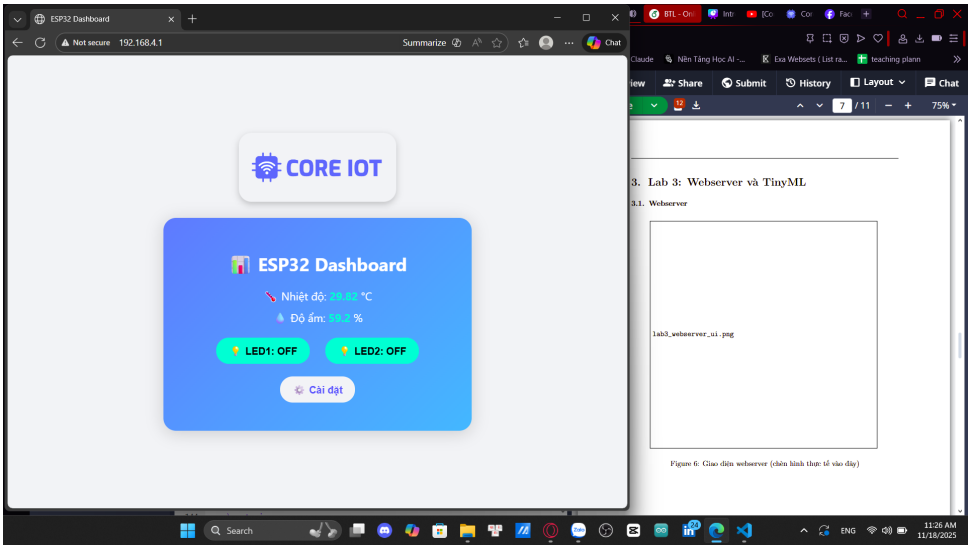

Figure 7: Giao diện webserver

3.3. Wifi Configuration

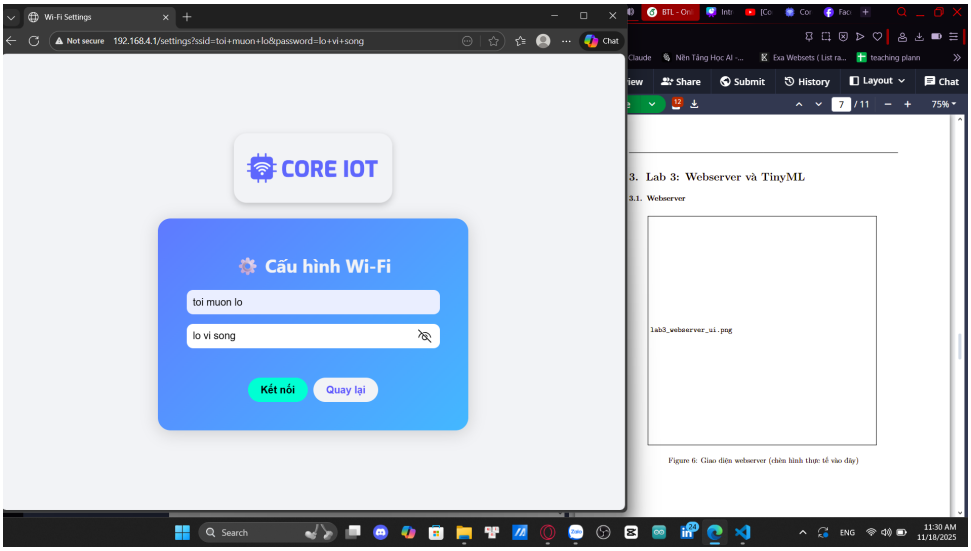

Figure 8: Giao diện kết nối wifi

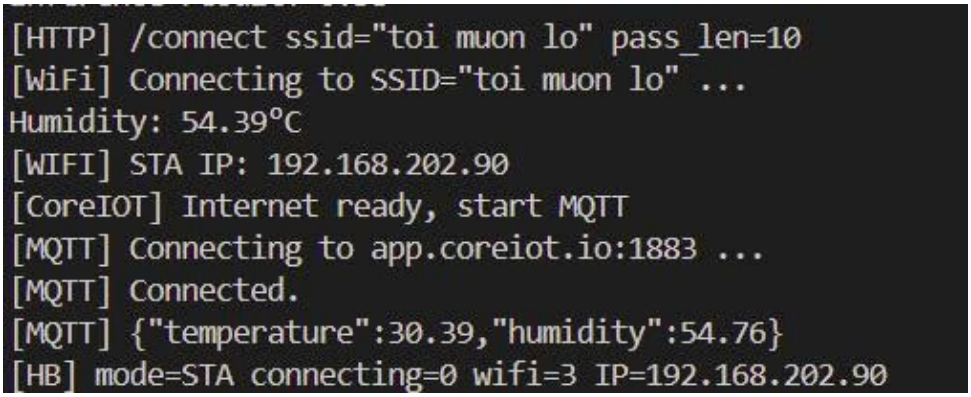

Figure 9: Sau khi bấm Kết nối

3.4. TinyML

TinyML Inference Logic

The TinyML task runs the TensorFlow Lite Micro model and retrieves a single output value from the interpreter. The logic is straightforward: the system reads the model's result and prints it to the serial console, without performing any further decision-making or actions.

```
float result = output->data.f[0];
Serial.print("Inference result: ");
Serial.println(result);
```

In the current implementation, the inference result is only logged. No threshold, alert mechanism, or conditional behavior is applied to the output. Future extensions may use this result to detect anomalies or trigger automatic responses.

Figure 10: Result

4. Lab 4: Semaphore, MQTT Broker và CoreIoT Gateway

4.1. Semaphore trong hệ thống

Semaphore Usage

We use a **binary semaphore** `xBinarySemaphoreInternet` to synchronize the WiFi task and the CoreIOT (MQTT) task.

- **WiFi task**: After the device successfully connects to the internet, it *gives* the semaphore to signal that the network is ready.
- **CoreIOT task**: On startup, it *takes* the semaphore and only begins the MQTT connection after the internet is available.

```
// ====== coreiot_task ch nh ======
void coreiot_task(void *pvParameters)
{
  //    i    WiFi ss
  Serial.println("[CoreIOT] Waiting for Internet.");
  xSemaphoreTake(xBinarySemaphoreInternet, portMAX_DELAY);
  Serial.println("[CoreIOT] Internet ready, start MQTT");


  mqtt.setServer(CORE_IOT_SERVER.c_str(), CORE_IOT_PORT.toInt());
  mqtt.setCallback(mqttCallback);
  ...
}}
```

Listing 3: Semaphore

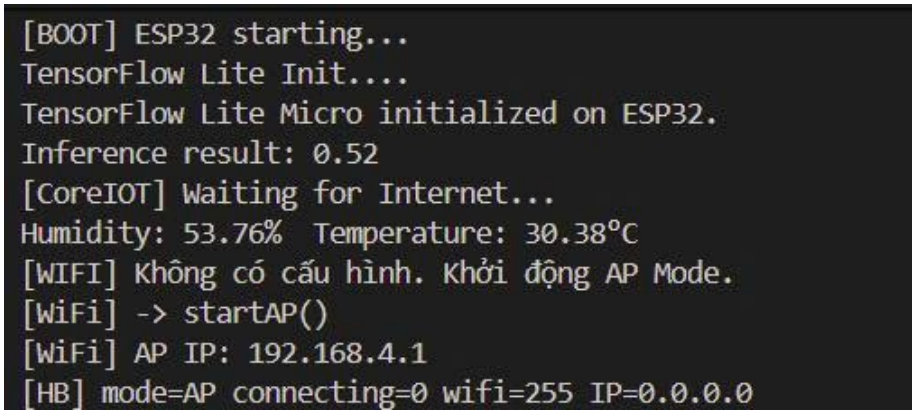

Figure 11: Sử dụng semaphore trong hệ thống

4.2. MQTT Broker

MQTT Broker Configuration Code

The following code snippet illustrates a simple MQTT Broker using the `HBMQTT` library. The broker listens on port 1883 and allows anonymous connections.

```
broker_config = {
    'listeners': {
        'default': {
            'type': 'tcp',
            'bind': '0.0.0.0:1883'
        }
    },
    'auth': {
        'allow-anonymous': True
    }
}

async def start_broker():
    broker = Broker(broker_config)
    await broker.start()
    print("MQTT Broker started...")
```

The broker serves as the communication hub, enabling ESP32 devices, Python publishers, and other clients to exchange data through MQTT topics.

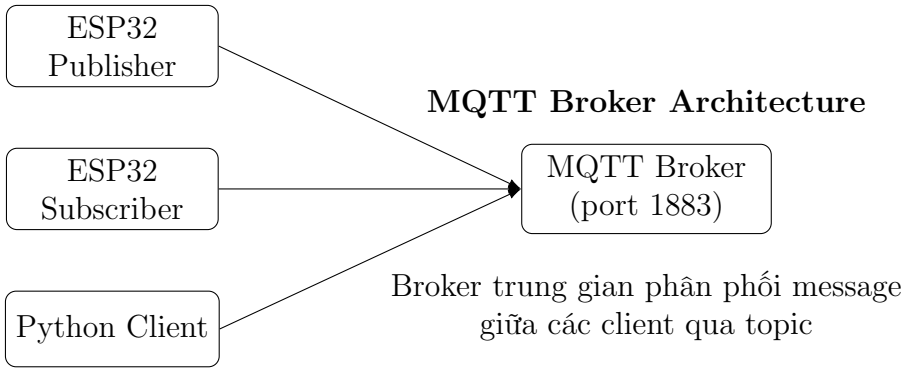

Figure 12: Kiến trúc tổng quát MQTT Broker

4.3. CoreIoT Gateway

MQTT Gateway Publishing Code

The following example demonstrates how a gateway publishes aggregated telemetry from multiple devices to an IoT backend.

```
client = mqtt.Client()
client.username_pw_set(ACCESS_TOKEN)
client.connect(THINGSBOARD_HOST, THINGSBOARD_PORT, 60)
client.loop_start()

telemetry = {
    "ESP32_001": [
        {"ts": int(time.time() * 1000),
         "values": {"temperature": 22.5, "humidity": 55}}
    ],
    "ESP32_002": [
        {"ts": int(time.time() * 1000),
         "values": {"temperature": 30.5, "humidity": 80}}
    ]
}

payload = json.dumps(telemetry)
client.publish('v1/gateway/telemetry', payload)
```

The gateway enables scalable IoT systems by consolidating telemetry from multiple nodes and forwarding it through a single MQTT connection, improving efficiency and reducing server-side load.

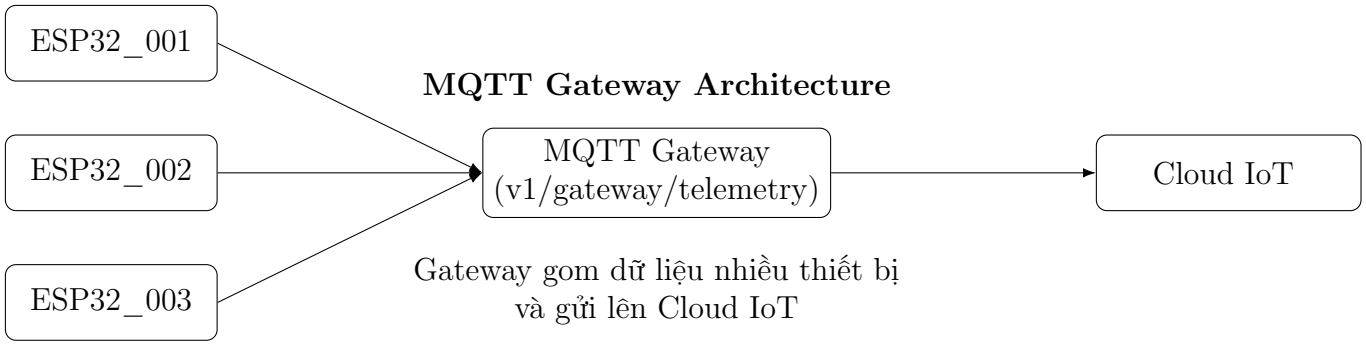

Figure 13: Kiến trúc MQTT Gateway

5. Source Code

Github Link: My Source Code